

HƯỚNG DẪN THỰC HIỆN PROJECT MÔN AI AGENT

ĐỀ TÀI SỐ 2

TRỢ LÝ TUÂN THỦ & PHÁP LÝ DOANH NGHIỆP (Compliance / Legal Assistant)

Kỹ thuật RAG nâng cao: Corrective RAG (CRAG)

Hướng dẫn chuẩn bị database - viết code - thực nghiệm - đánh giá - làm báo cáo

Tài liệu tham chiếu: “Đề xuất Project môn Agent” và “Hướng dẫn thực hiện Project môn Agent - Bản chi tiết”

MỤC LỤC HƯỚNG DẪN

- 1. Xác định phạm vi và mục tiêu đề tài
- 2. Kiến trúc tổng thể của hệ thống
- 3. Stack công nghệ đề xuất
- 4. Chuẩn bị môi trường
- 5. Tổ chức thư mục project
- 6. Chuẩn bị corpus pháp luật
- 7. Thiết kế database
- 8. Tạo database bằng Python
- 9. Cấu hình LLM và embedding
- 10. Định nghĩa Agent State
- 11. Nạp dữ liệu vào Chroma
- 12. Xây dựng các Tools
- 13. Xây dựng CRAG relevance grader
- 14. Giải thích RAG và Corrective RAG
- 15. Retrieve node
- 16. Web fallback
- 17. Database query
- 18. Prompt sinh câu trả lời
- 19. Xây dựng LangGraph
- 20. Chương trình chạy thử main.py
- 21. Ba test bắt buộc
- 22. Baseline RAG so với CRAG
- 23. Thiết kế test-set 40 câu
- 24. Đánh giá bằng RAGAS
- 25. Chỉ số Fallback Accuracy
- 26. Thí nghiệm ngưỡng relevance
- 27-34. Cấu trúc báo cáo kỹ thuật
- 35. Checklist để đạt điểm tốt
- 36. Kế hoạch triển khai theo giai đoạn

Khuyến nghị: Thứ tự triển khai nên là: DATABASE → RAG → CRAG → LANGGRAPH → TEST → EVALUATION → STREAMLIT → REPORT.

1. Xác định phạm vi và mục tiêu đề tài

Tên đề tài đề xuất: AI Agent hỗ trợ tra cứu và kiểm tra tuân thủ pháp lý doanh nghiệp Việt Nam sử dụng Corrective RAG.

Theo đề bài, Project 2 yêu cầu xây dựng trợ lý tuân thủ/pháp lý doanh nghiệp, trong đó Corrective RAG được dùng để chấm độ liên quan của tài liệu truy hồi và fallback sang web search khi corpus không đủ. Hệ thống cần kết hợp web search, database quan hệ, RAG, LangGraph và LLM chạy nội bộ qua Ollama.

1.1. Phạm vi nên thu hẹp

Không nên xây một trợ lý trả lời toàn bộ pháp luật Việt Nam. Phạm vi quá rộng sẽ làm khó thu thập dữ liệu, khó tạo ground-truth và khó đánh giá định lượng.

- Đăng ký doanh nghiệp.
- Pháp luật lao động.
- Bảo hiểm xã hội.
- Thuế doanh nghiệp.
- An toàn lao động.

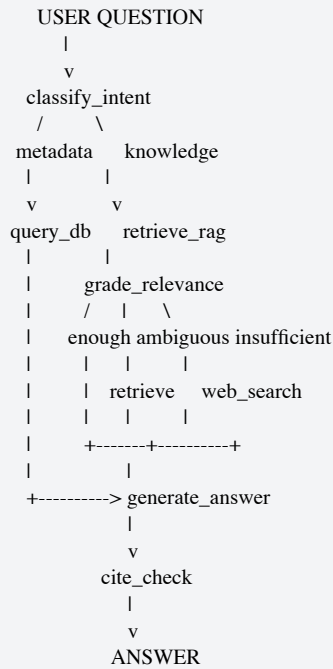
Phạm vi khuyến nghị: Để dễ hoàn thành và dễ demo, có thể tập trung vào hai nhóm: đăng ký doanh nghiệp + lao động.

1.2. Các câu hỏi mẫu hệ thống phải xử lý

- Điều kiện thành lập công ty TNHH là gì?
- Văn bản X hiện còn hiệu lực không?
- Quy định hiện hành về hợp đồng lao động là gì?
- Quy định vừa được sửa đổi nhưng kho dữ liệu nội bộ chưa cập nhật thì hệ thống xử lý thế nào?

2. Kiến trúc tổng thể của hệ thống

Lưu ý cốt lõi của đề bài là retrieve → grade-relevance → đủ thì generate, thiếu thì web-search → generate. Để thể hiện rõ tính Agent, có thể mở rộng router cho câu hỏi metadata/database.



2.1. Vai trò của từng khối

Khối	Vai trò
Router	Phân loại câu hỏi cần tra tri thức hay metadata/database.
Retriever	Tìm các đoạn văn bản pháp luật liên quan trong vector database.
Relevance grader	Chấm mức liên quan của tài liệu so với câu hỏi.
Web search	Bổ sung dữ liệu khi corpus thiếu hoặc lỗi thời.
Database tool	Tra số hiệu, ngày ban hành, trạng thái hiệu lực và metadata.
Generator	Sinh câu trả lời từ evidence đã được kiểm tra.
Citation check	Đảm bảo câu trả lời gắn số hiệu văn bản và điều khoản.

3. Stack công nghệ đề xuất

Thành phần	Công nghệ đề xuất
LLM	Ollama - qwen2.5:7b
Embedding	Ollama - bge-m3
Agent orchestration	LangGraph
Vector Database	Chroma
Relational Database	SQLite
Web search	DuckDuckGo Search
Evaluation	RAGAS
Trace	LangFuse
Ngôn ngữ	Python
Giao diện demo	Streamlit - làm sau khi Agent chạy ổn định

Lưu ý: Nếu máy đủ mạnh, có thể thay qwen2.5:7b bằng mô hình lớn hơn. Tuy nhiên để bảo đảm demo ổn định, bản 7B là lựa chọn hợp lý.

4. BƯỚC 1 - Chuẩn bị môi trường

Cài Python 3.11 và Ollama trước. Sau đó tải hai model:

```
ollama pull qwen2.5:7b
ollama pull bge-m3
```

Tạo môi trường ảo:

```
python -m venv .venv
```

Kích hoạt trên Windows:

```
.venv\Scripts\activate
```

Cài các thư viện:

```

pip install langgraph
pip install langchain
pip install langchain-ollama
pip install langchain-chroma
pip install langchain-community
pip install langchain-text-splitters

pip install chromadb
pip install duckduckgo-search
pip install pypdf
pip install beautifulsoup4
pip install pandas

pip install ragas
pip install datasets
pip install langfuse
pip install streamlit

```

5. BƯỚC 2 - Tổ chức thư mục project

```

legal_crag_agent/
├── data/
│   ├── raw/
│   │   ├── vanban01.pdf
│   │   ├── vanban02.pdf
│   │   └── ...
│   └── processed/
├── chroma/
├── state.py
├── llm.py
├── create_db.py
├── ingest.py
├── tools.py
├── nodes.py
├── graph.py
├── main.py
├── eval.py
├── app.db
└── requirements.txt

```

Cấu trúc này giữ đúng tinh thần skeleton của đề bài: tách state, model, ingest, tools, nodes, graph và evaluation thành các file độc lập.

6. BƯỚC 3 - Chuẩn bị corpus pháp luật

Corpus nên gồm văn bản pháp luật Việt Nam có liên quan trực tiếp tới phạm vi đề tài. Không cần tải hàng nghìn văn bản; một corpus nhỏ nhưng được làm sạch và có metadata tốt sẽ dễ đánh giá hơn.

Thành phần	Quy mô gợi ý
Số văn bản	50-100 văn bản
Số điều khoản	Khoảng 500-1.500

Thành phần	Quy mô gợi ý
Số chunks sau khi chia	Khoảng 1.000-3.000

6.1. Metadata tối thiểu

- id
- so_hieu
- loai
- co_quan
- ngay_ban_hanh
- trang_thai
- ten_van_ban
- nguon

6.2. Ví dụ một bản ghi metadata

```
VB001
01/2021/NĐ-CP
Nghị định
Chính phủ
2021-01-04
con_hieu_luc
Đăng ký doanh nghiệp
<URL nguồn chính thức>
```

Quan trọng: Không nên tự tạo nội dung giả lập. Corpus demo nên lấy từ văn bản pháp lý thực tế và lưu rõ nguồn.

7. BƯỚC 4 - Thiết kế database

Schema tối thiểu theo đề bài:

```
CREATE TABLE documents (
  id TEXT PRIMARY KEY,
  so_hieu TEXT,
  loai TEXT,
  co_quan TEXT,
  ngay_ban_hanh DATE,
  trang_thai TEXT
);

CREATE TABLE articles (
  doc_id TEXT,
  dieu INTEGER,
  tieu_de TEXT,
  noi_dung TEXT
);

CREATE INDEX idx_art_doc
ON articles(doc_id);
```

7.1. Schema mở rộng khuyến nghị

```
CREATE TABLE documents (
  id TEXT PRIMARY KEY,
  so_hieu TEXT UNIQUE,
  ten_van_ban TEXT,
  loai TEXT,
  co_quan TEXT,
  ngay_ban_hanh DATE,
  ngay_hieu_luc DATE,
  trang_thai TEXT,
  thay_the_boi TEXT,
  url TEXT
);

CREATE TABLE articles (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  doc_id TEXT,
  dieu INTEGER,
  tieu_de TEXT,
  noi_dung TEXT,
  FOREIGN KEY(doc_id) REFERENCES documents(id)
);

CREATE INDEX idx_doc_sohieu
ON documents(so_hieu);

CREATE INDEX idx_doc_status
ON documents(trang_thai);

CREATE INDEX idx_art_doc
ON articles(doc_id);
```

Trường thay_the_boi hỗ trợ xử lý bài toán văn bản cũ/mới - một thách thức quan trọng của đề tài.

8. BƯỚC 5 - Viết chương trình tạo database

File: create_db.py

```

import sqlite3

conn = sqlite3.connect("app.db")
cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS documents (
    id TEXT PRIMARY KEY,
    so_hieu TEXT UNIQUE,
    ten_van_ban TEXT,
    loai TEXT,
    co_quan TEXT,
    ngay_ban_hanh DATE,
    ngay_hieu_luc DATE,
    trang_thai TEXT,
    thay_the_boi TEXT,
    url TEXT
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS articles (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    doc_id TEXT,
    dieu INTEGER,
    tieu_de TEXT,
    noi_dung TEXT,
    FOREIGN KEY(doc_id) REFERENCES documents(id)
)
""")

cur.execute("""
CREATE INDEX IF NOT EXISTS idx_art_doc
ON articles(doc_id)
""")

conn.commit()
conn.close()

print("Database created.")

```

Chạy:

```
python create_db.py
```

Sau khi chạy thành công, trong thư mục project phải xuất hiện file app.db.

9. BƯỚC 6 - Cấu hình LLM và embedding

File: llm.py

```

from langchain_ollama import ChatOllama, OllamaEmbeddings

llm = ChatOllama(
    model="qwen2.5:7b",
    temperature=0
)

embeddings = OllamaEmbeddings(
    model="bge-m3"
)

```

temperature=0 phù hợp với bài toán pháp lý vì cần câu trả lời ổn định, giảm tính ngẫu nhiên.

10. BƯỚC 7 - Định nghĩa Agent State

File: state.py

```
from typing import TypedDict

class AgentState(TypedDict, total=False):
    query: str
    documents: list[dict]
    db_results: list[dict]
    web_results: list[dict]
    grades: dict
    route: str
    iteration: int
    generation: str
```

State là bộ nhớ làm việc của LangGraph. Mỗi node đọc một số trường và cập nhật một số trường khác.

11. BƯỚC 8 - Nạp dữ liệu vào Chroma

Không embedding nguyên một văn bản dài thành một vector. Nên chia theo đầu khoản hoặc chunk nhỏ.

```
Văn bản
├─ Đầu 1 → Chunk 1
├─ Đầu 2 → Chunk 2
└─ Đầu 3 → Chunk 3
```

File: ingest.py

```

from langchain_chroma import Chroma
from langchain_text_splitters import RecursiveCharacterTextSplitter
from llm import embeddings

def build_index(docs):
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=700,
        chunk_overlap=100
    )

    texts = []
    metas = []

    for doc in docs:
        chunks = splitter.split_text(doc["text"])

        for chunk in chunks:
            texts.append(chunk)

            metas.append({
                "doc_id": doc["doc_id"],
                "so_hieu": doc["so_hieu"],
                "dieu": doc["dieu"],
                "nguồn": doc.get("nguồn", "")
            })

    Chroma.from_texts(
        texts=texts,
        embedding=embeddings,
        metadatas=metas,
        persist_directory="/chroma",
        collection_name="legal_corpus"
    )

```

Mẹo kỹ thuật: Giữ metadata ở từng chunk để sau này câu trả lời có thể trích số hiệu, điều và nguồn.

12. BƯỚC 9 - Xây dựng các Tools

12.1. Tool 1 - RAG retriever

```

from langchain_chroma import Chroma
from llm import embeddings

vectorstore = Chroma(
    collection_name="legal_corpus",
    embedding_function=embeddings,
    persist_directory="/chroma"
)

retriever = vectorstore.as_retriever(
    search_kwargs={"k": 5}
)

```

12.2. Tool 2 - Database

```
import sqlite3

def run_sql(sql):
    con = sqlite3.connect(
        "file:app.db?mode=ro",
        uri=True
    )

    con.row_factory = sqlite3.Row
    rows = con.execute(sql).fetchall()
    con.close()

    return [dict(row) for row in rows]
```

Mở database ở chế độ read-only giúp giảm rủi ro khi sau này tích hợp Text-to-SQL.

12.3. Tool 3 - Web search

```
from langchain_community.tools import DuckDuckGoSearchResults

web_search = DuckDuckGoSearchResults(
    output_format="list"
)
```

13. BƯỚC 10 - Xây dựng CRAG relevance grader

Đây là node quan trọng nhất của đề tài. Nếu bỏ relevance grader, hệ thống chỉ còn là RAG thông thường.

```

import json
from llm import llm

GRADER_PROMPT = """
Bạn là bộ đánh giá tài liệu pháp lý.

Chấm mức độ liên quan giữa TÀI LIỆU và CÂU HỎI
trên thang điểm từ 0 đến 1.

Chỉ trả JSON:
{
  "score": 0.0,
  "ly_do": "..."
}

TÀI LIỆU:
{doc}

CÂU HỎI:
{q}
"""

def grade_relevance(state):
    docs = state["documents"]
    kept = []
    scores = []

    for d in docs:
        prompt = GRADER_PROMPT.format(
            doc=d["text"][:1500],
            q=state["query"]
        )

        response = llm.invoke(prompt)

        try:
            result = json.loads(response.content)
            score = float(result.get("score", 0))
        except Exception:
            score = 0

        scores.append(score)

        if score >= 0.5:
            kept.append(d)

    if len(kept) == 0:
        verdict = "insufficient"
    elif len(kept) < len(docs) / 2:
        verdict = "ambiguous"
    else:
        verdict = "enough"

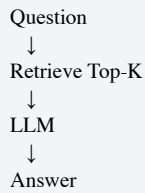
    return {
        "documents": kept or docs,
        "grades": {
            "verdict": verdict,
            "scores": scores
        },
        "iteration": state.get("iteration", 0) + 1
    }

```

Điểm nghiên cứu: Ban đầu dùng threshold = 0.5 theo skeleton đề bài. Sau đó nên chạy thực nghiệm với 0.3 / 0.5 / 0.7 để chứng minh ngưỡng phù hợp.

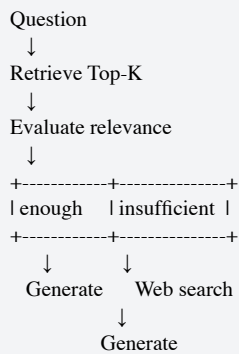
14. RAG thông thường và Corrective RAG

14.1. Traditional RAG



Nhược điểm: nếu Retriever trả về tài liệu sai hoặc kém liên quan, LLM vẫn có thể dùng chúng và tạo câu trả lời thiếu chính xác.

14.2. Corrective RAG



Điểm khác biệt cốt lõi: Agent không mặc định tin kết quả retrieval. Nó tự đánh giá chất lượng evidence trước khi quyết định bước tiếp theo.

15. BƯỚC 11 - Retrieve node

Đặt trong nodes.py:

```

from tools import retriever

def retrieve_docs(state):
    docs = retriever.invoke(
        state["query"]
    )

    results = []

    for d in docs:
        results.append({
            "text": d.page_content,
            "meta": d.metadata
        })

    return {
        "documents": results
    }
  
```

16. BƯỚC 12 - Web fallback

```
from tools import web_search

def run_web_search(state):
    results = web_search.invoke(
        state["query"]
    )

    return {
        "web_results": results
    }
```

16.1. Case demo nên chuẩn bị

```
Corpus chỉ cập nhật đến thời điểm T1
↓
Người dùng hỏi quy định mới ở thời điểm T2
↓
RAG không tìm thấy evidence đủ tốt
↓
CRAG grader = insufficient
↓
Agent gọi web_search
↓
Sinh câu trả lời từ evidence mới
```

17. BƯỚC 13 - Database query

Câu hỏi về trạng thái văn bản là dạng structured data, không cần vector search.

Ví dụ câu hỏi: “Văn bản X còn hiệu lực không?”

```
SELECT
    so_hieu,
    ten_van_ban,
    ngay_ban_hanh,
    ngay_hieu_luc,
    trang_thai,
    thay_the_boi
FROM documents
WHERE so_hieu = ?;
```

Khuyến nghị: Trong bản demo đầu tiên, có thể dùng query parameter cố định thay vì cho LLM tự sinh SQL. Sau khi hệ thống ổn định mới thêm Text-to-SQL.

18. BƯỚC 14 - Prompt sinh câu trả lời

```
ANSWER_PROMPT = """
Bạn là trợ lý hỗ trợ tra cứu tuân thủ pháp lý doanh nghiệp.

Chỉ trả lời dựa trên NGŨ CẢNH được cung cấp.

Yêu cầu:
1. Không tự suy đoán quy định pháp luật.
2. Với mỗi kết luận phải chỉ rõ:
   - số hiệu văn bản;
   - điều khoản;
   - nguồn.
3. Nếu bằng chứng chưa đủ, phải ghi:
   "Chưa đủ căn cứ để kết luận."
4. Nếu các nguồn có mâu thuẫn, ưu tiên văn bản
   còn hiệu lực và nêu rõ sự khác biệt.

NGŨ CẢNH:
{context}

CÂU HỎI:
{query}
"""
```

Prompt này giữ đúng yêu cầu của đề câu trả lời phải dựa trên trích dẫn, không tự suy đoán và nêu rõ khi chưa đủ căn cứ.

19. BƯỚC 15 - Xây dựng LangGraph

File: graph.py

```

from langgraph.graph import StateGraph, END
from state import AgentState

from nodes import (
    retrieve_docs,
    grade_relevance,
    run_web_search,
    generate_answer
)

MAX_ITER = 2

def decide_after_grade(state):
    verdict = state["grades"][0]["verdict"]

    if verdict == "enough":
        return "generate"

    if state.get("iteration", 0) >= MAX_ITER:
        return "generate"

    if verdict == "insufficient":
        return "web"

    return "retrieve"

graph = StateGraph(AgentState)

graph.add_node("retrieve", retrieve_docs)
graph.add_node("grade", grade_relevance)
graph.add_node("web", run_web_search)
graph.add_node("generate", generate_answer)

graph.set_entry_point("retrieve")

graph.add_edge("retrieve", "grade")

graph.add_conditional_edges(
    "grade",
    decide_after_grade,
    {
        "generate": "generate",
        "web": "web",
        "retrieve": "retrieve"
    }
)

graph.add_edge("web", "generate")
graph.add_edge("generate", END)

app = graph.compile()

```

MAX_ITER giúp chặn vòng lặp vô hạn khi grader liên tục trả ambiguous.

20. BƯỚC 16 - Chương trình chạy thử main.py

```
from graph import app

while True:
    question = input(
        "\nCâu hỏi pháp lý: "
    )

    if question.lower() in [
        "exit",
        "quit"
    ]:
        break

    result = app.invoke({
        "query": question,
        "iteration": 0
    })

    print("\nTRẢ LỜI:\n")
    print(result["generation"])
```

Chạy:

```
python main.py
```

21. Ba test bắt buộc nên demo

Test	Câu hỏi	Luồng kỳ vọng
1 - In corpus	Điều kiện đăng ký doanh nghiệp theo nghị định hiện hành là gì?	RAG → relevance cao → enough → generate
2 - Out of corpus	Quy định vừa sửa đổi gần đây có gì mới?	RAG → relevance thấp → insufficient → web_search → generate
3 - Database	Văn bản X còn hiệu lực không?	DB → documents.trang_thai → answer

Demo: Test 2 là test quan trọng nhất để chứng minh hệ thống thực sự sử dụng Corrective RAG.

22. Nâng mức Thạc sĩ: Baseline RAG so với CRAG

Không nên chỉ chứng minh “Agent chạy được”. Nên thiết kế thí nghiệm để so sánh với baseline.

Hệ thống	Cấu hình
System A - Traditional RAG	Retrieve Top-K → Generate
System B - Corrective RAG	Retrieve Top-K → Grade relevance → Web fallback khi cần → Generate

Giữ giống nhau giữa hai hệ thống:

- LLM
- Embedding model
- Dataset
- Test questions
- Top-K

Chỉ để CRAG khác ở:

- Relevance grader
- Conditional routing
- Web fallback

23. Thiết kế test-set khoảng 40 câu

Nhóm câu hỏi	Số lượng
Trong corpus	15
Ngoài corpus	10
Kiểm tra hiệu lực DB	5
Văn bản cũ/mới mâu thuẫn	5
Câu hỏi thiếu thông tin	5
Tổng	40

Mỗi record có thể lưu dạng JSON:

```
{
  "question": "...",
  "type": "in_corpus",
  "expected_source": "VB001",
  "expected_article": 5,
  "ground_truth": "..."
}
```

24. Đánh giá bằng RAGAS

Bộ khung đề bài gợi ý các metric: faithfulness, answer relevancy, context precision và context recall.

Metric	RAG	CRAG
Faithfulness	0.xx	0.xx
Answer relevancy	0.xx	0.xx
Context precision	0.xx	0.xx
Context recall	0.xx	0.xx

Metric	RAG	CRAg
Citation accuracy	xx%	xx%
Web fallback accuracy	-	xx%

Nguyên tắc báo cáo: Không điền số giả. Chỉ đưa các giá trị sau khi đã chạy thực nghiệm.

25. Chỉ số nên bổ sung: Fallback Accuracy

25.1. Fallback Accuracy

Số câu out-of-corpus được CRAg nhận diện đúng là insufficient

Tổng số câu out-of-corpus

25.2. False Fallback Rate

Số câu in-corpus nhưng Agent vẫn gọi web

Tổng số câu in-corpus

Hai metric này đo trực tiếp chất lượng quyết định của CRAg, đặc biệt hữu ích khi phân tích threshold.

26. Thí nghiệm ngưỡng relevance

Thử ba ngưỡng:

threshold = 0.3
threshold = 0.5
threshold = 0.7

Threshold	Correct fallback	False fallback	Accuracy
0.3	...	...	...
0.5	...	...	...
0.7	...	...	...

Sau thực nghiệm, nhóm kết luận ngưỡng nào cân bằng tốt nhất giữa bỏ sót evidence và gọi web quá nhiều.

27. Cấu trúc báo cáo - Chương 1: Giới thiệu

- 1.1 Bối cảnh
- 1.2 Vấn đề
- 1.3 Mục tiêu

- 1.4 Phạm vi
- 1.5 Đóng góp của project

Đoạn mục tiêu có thể viết:

Xây dựng một AI Agent hỗ trợ tra cứu tuân thủ pháp lý doanh nghiệp bằng LangGraph và mô hình ngôn ngữ chạy nội bộ Ollama, kết hợp cơ sở dữ liệu quan hệ, RAG và web search. Hệ thống sử dụng Corrective RAG để đánh giá mức độ liên quan của bằng chứng được truy hồi và tự động chuyển sang nguồn thông tin bên ngoài khi corpus nội bộ không cung cấp đủ căn cứ.

28. Chương 2 - Cơ sở lý thuyết

- 2.1 Large Language Model
- 2.2 AI Agent
- 2.3 Retrieval-Augmented Generation
- 2.4 Vector Embedding
- 2.5 Vector Database
- 2.6 LangGraph
- 2.7 Corrective RAG
- 2.8 RAGAS

Trọng tâm: Phần quan trọng nhất của Chương 2 là so sánh Traditional RAG và Corrective RAG.

29. Chương 3 - Phân tích và thiết kế

- 3.1 Yêu cầu chức năng
- 3.2 Yêu cầu phi chức năng
- 3.3 Kiến trúc hệ thống
- 3.4 LangGraph workflow
- 3.5 Thiết kế database
- 3.6 Vector database
- 3.7 Thiết kế Agent State
- 3.8 Thiết kế tools

Nên có ít nhất ba hình/sơ đồ

- System Architecture
- LangGraph Workflow
- ERD: documents 1:N articles

30. Chương 4 - Chuẩn bị dữ liệu

- 4.1 Nguồn dữ liệu

- 4.2 Thu thập dữ liệu
- 4.3 Làm sạch
- 4.4 Tách đi đầu khoản
- 4.5 Chunking
- 4.6 Metadata
- 4.7 Embedding
- 4.8 Xây dựng Chroma
- 4.9 Xây dựng SQLite

```
chunk_size = 700
chunk_overlap = 100
embedding = bge-m3
```

31. Chương 5 - Xây dựng AI Agent

- 5.1 Ollama
- 5.2 Retriever
- 5.3 Database Tool
- 5.4 Web Search Tool
- 5.5 Relevance Grader
- 5.6 Corrective RAG
- 5.7 Conditional Routing
- 5.8 Answer Generation
- 5.9 Citation Control
- 5.10 LangFuse Trace

32. Chương 6 - Thực nghiệm

- 6.1 Dataset thử nghiệm
- 6.2 Baseline RAG
- 6.3 CRAG
- 6.4 Evaluation metrics
- 6.5 Thử threshold
- 6.6 RAGAS

Chương này giúp project vượt khỏi mức một chatbot demo và trở thành một bài thực nghiệm kỹ thuật phù hợp trình độ cao học.

33. Chương 7 - Kết quả và thảo luận

- 7.1 Kết quả retrieval
- 7.2 Kết quả CRAG
- 7.3 Kết quả web fallback
- 7.4 RAGAS
- 7.5 So sánh RAG và CRAG
- 7.6 Phân tích lỗi

Nên đưa một case lỗi thật:

```

Query
↓
RAG retrieve sai điều khoản
↓
CRAG relevance = 0.24
↓
Agent loại tài liệu
↓
Web search
↓
Lấy evidence mới
↓
Answer có trích ngữ ần

```

34. Chương 8 - Hạn chế và hướng phát triển

34.1. Hạn chế

- Corpus còn nhỏ.
- Web search chưa luôn đảm bảo ngữ ần chính thống.
- LLM 7B có giới hạn reasoning.
- Grader dựa vào LLM nên vẫn có thể ch ấ m sai.
- Chưa xử lý đ ầy đủ quan hệ sửa đổi / thay thế / bãi bỏ.
- Chưa có temporal legal reasoning hoàn chỉnh.

34.2. Hướng phát triển

- Knowledge Graph pháp lý.
- Hybrid Retrieval.
- Reranking.
- Temporal RAG.
- Legal citation verification.
- Document versioning.
- Human-in-the-loop.

35. Checklist để đạt điểm tốt

- ☐ Ollama chạy local.
- ☐ qwen2.5:7b.
- ☐ bge-m3.
- ☐ SQLite.
- ☐ Chroma.
- ☐ LangGraph.
- ☐ RAG.
- ☐ Corrective RAG.
- ☐ Database tool.
- ☐ Web search tool.
- ☐ Conditional routing.
- ☐ Citation.
- ☐ Khoảng 40 test questions.
- ☐ RAGAS.
- ☐ LangFuse trace.
- ☐ Baseline RAG vs CRAG.
- ☐ Demo Streamlit.
- ☐ Báo cáo kỹ thuật.

36. Kế hoạch triển khai theo giai đoạn

Giai đoạn	Công việc	Sản phẩm
1	Chốt phạm vi pháp lý	Danh sách văn bản
2	Thu thập 50-100 văn bản	PDF/TXT
3	Làm sạch + metadata	CSV/JSON
4	Tạo SQLite	app.db
5	Tạo Chroma	chroma/
6	Test RAG cơ bản	Retrieval chạy
7	Viết relevance grader	CRAG chạy
8	Tích hợp web search	Fallback chạy
9	Xây LangGraph	Agent hoàn chỉnh
10	Tạo 40 test questions	test_set.json
11	RAG vs CRAG	Bảng kết quả
12	RAGAS + LangFuse	Metrics + trace
13	Streamlit	Demo
14	Viết báo cáo	Word/PDF
15	Làm slide	15-20 slide

36.1. Thứ tự thực hiện cuối cùng

```

DATABASE
↓
RAG
↓
CRAG
↓
LANGGRAPH
↓
TEST
↓
EVALUATION
↓
STREAMLIT
↓
REPORT

```

Khuyến nghị quan trọng: Không nên làm Streamlit ngay từ đầu. Hãy làm cho database, RAG, CRAG và LangGraph chạy đúng trước; giao diện chỉ là lớp cuối cùng.

37. BỔ SUNG - ACCOUNT VÀ USER MEMORY

Để Agent có khả năng phục vụ nhiều người dùng và tư vấn tốt hơn khi người dùng quay lại, nên bổ sung lớp Account + Memory. Hệ thống sẽ tạo tài khoản riêng cho từng người dùng, lưu lịch sử câu hỏi/phiên làm việc, trích xuất một số thông tin bền vững có ích và nạp lại ngữ cảnh này ở những lần đăng nhập sau.

Khái niệm quan trọng: “Memory” trong project này không có nghĩa là huấn luyện lại trọng số của LLM. Đây là cơ chế lưu trữ và truy hồi lịch sử/ngữ cảnh từ database rồi đưa vào prompt hoặc Agent State. Cách này để kiểm soát, dễ đánh giá và phù hợp hơn với project môn học.

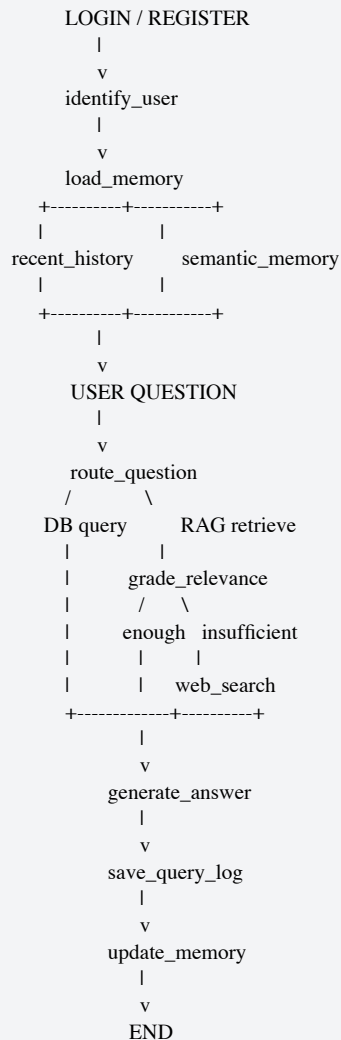
37.1. Mục tiêu của phần Memory

- Mỗi người dùng có account riêng và dữ liệu không bị trộn với người dùng khác.
- Lưu lại câu hỏi, câu trả lời, route của Agent và thời gian truy vấn.
- Khi người dùng quay lại, Agent có thể đọc một phần lịch sử gần đây để hiểu ngữ cảnh.
- Lưu một số “semantic memory” bền vững như loại hình doanh nghiệp, lĩnh vực thường quan tâm hoặc phạm vi tư vấn đã chọn.
- Cho phép đánh giá được khả năng cá nhân hóa và độ chính xác của memory retrieval.

37.2. Ba lớp Memory nên phân biệt

Loại memory	Lưu ở đâu	Mục đích
Working / Session memory	AgentState / session	Giữ ngữ cảnh của phiên đang chạy.
Episodic memory	query_logs	Lưu lịch sử câu hỏi, câu trả lời và các lần Agent đã xử lý.
Semantic / Profile memory	user_memories	Lưu thông tin bền vững đã được xác nhận hoặc trích xuất từ lịch sử.

38. Kiến trúc Agent sau khi bổ sung Memory



Điểm mới là memory được nạp trước khi định tuyến câu hỏi và được cập nhật sau khi câu trả lời đã hoàn tất. Tuy nhiên memory chỉ cung cấp bối cảnh cá nhân hóa; các kết luận pháp lý vẫn phải dựa vào RAG/DB/web evidence hiện hành.

Guardrail pháp lý: Không dùng câu trả lời cũ trong query_logs như một nguồn pháp luật. Lịch sử chỉ giúp hiểu “người dùng đang nói về trường hợp nào”; mọi quy định pháp lý phải được kiểm chứng lại từ corpus, database hiệu lực hoặc web search.

38.1. Cấu trúc thư mục sau khi mở rộng

```

legal_crag_agent/
├── data/
├── chroma/
├── state.py
├── llm.py
├── create_db.py
├── ingest.py
├── tools.py
├── nodes.py
├── graph.py
├── auth.py      # đăng ký / đăng nhập
├── memory.py    # query log + user memory
├── app.py       # Streamlit UI
├── main.py
├── eval.py
├── app.db
└── requirements.txt

```

39. Thiết kế database cho Account, Query Log và Memory

Có thể tiếp tục dùng chung SQLite app.db. Khóa user_id phải xuất hiện ở mọi bảng lưu lịch sử để bảo đảm tách biệt dữ liệu theo người dùng.

```

CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT UNIQUE NOT NULL,
  password_hash BLOB NOT NULL,
  full_name TEXT,
  memory_opt_in INTEGER DEFAULT 1,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP,
  last_login TEXT
);

CREATE TABLE chat_sessions (
  id TEXT PRIMARY KEY,
  user_id INTEGER NOT NULL,
  started_at TEXT DEFAULT CURRENT_TIMESTAMP,
  last_active_at TEXT DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY(user_id) REFERENCES users(id)
);

```

```

CREATE TABLE query_logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  session_id TEXT,
  question TEXT NOT NULL,
  answer TEXT,
  route TEXT,
  source_type TEXT,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY(user_id) REFERENCES users(id)
);

CREATE INDEX idx_query_user_time
ON query_logs(user_id, created_at);

```

```

CREATE TABLE user_memories (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  memory_key TEXT NOT NULL,
  memory_value TEXT NOT NULL,
  memory_type TEXT DEFAULT 'semantic',
  confidence REAL DEFAULT 1.0,
  updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(user_id, memory_key),
  FOREIGN KEY(user_id) REFERENCES users(id)
);

CREATE INDEX idx_memory_user
ON user_memories(user_id);

```

39.1. Ví dụ dữ liệu memory

user_id	memory_key	memory_value	Ý nghĩa
12	business_type	Công ty TNHH	Loại hình doanh nghiệp người dùng đang quan tâm.
12	province	Long An	Địa bàn thường dùng trong tình huống tư vấn.
12	frequent_topic	Hợp đồng lao động	Chủ đề pháp lý xuất hiện lặp lại.
12	preferred_answer	Tóm tắt + đi đầu khoản + ngu ần	Kiểu trình bày mong muốn.

40. Viết code tạo Account và đăng nhập

Cài thêm thư viện mã hóa mật khẩu:

```
pip install bcrypt
```

Bảo mật: Không lưu password dạng plain text. Chỉ lưu password_hash.

File: auth.py

```

import sqlite3
import bcrypt
from datetime import datetime

DB = "app.db"

def create_user(username, password, full_name=""):
    password_hash = bcrypt.hashpw(
        password.encode("utf-8"),
        bcrypt.gensalt()
    )

    with sqlite3.connect(DB) as con:
        cur = con.execute(
            """INSERT INTO users(username, password_hash, full_name)
               VALUES (?, ?, ?)""",
            (username, password_hash, full_name)
        )
        return cur.lastrowid

```

```

def authenticate(username, password):
    with sqlite3.connect(DB) as con:
        con.row_factory = sqlite3.Row
        row = con.execute(
            "SELECT * FROM users WHERE username = ?",
            (username,)
        ).fetchone()

        if not row:
            return None

        ok = bcrypt.checkpw(
            password.encode("utf-8"),
            row["password_hash"]
        )

        if not ok:
            return None

        con.execute(
            "UPDATE users SET last_login=? WHERE id=?",
            (datetime.now().isoformat(), row["id"])
        )
        return dict(row)

```

41. Viết module lưu lịch sử và truy hồi Memory

File: memory.py

```

import sqlite3

DB = "app.db"

def log_interaction(user_id, session_id, question,
                   answer, route, source_type):
    with sqlite3.connect(DB) as con:
        con.execute(
            """INSERT INTO query_logs
               (user_id, session_id, question, answer, route, source_type)
               VALUES (?, ?, ?, ?, ?, ?)""",
            (user_id, session_id, question,
             answer, route, source_type)
        )

```

```
def load_recent_history(user_id, limit=8):
    with sqlite3.connect(DB) as con:
        con.row_factory = sqlite3.Row
        rows = con.execute(
            """SELECT question, answer, route, created_at
               FROM query_logs
               WHERE user_id = ?
               ORDER BY id DESC
               LIMIT ?""",
            (user_id, limit)
        ).fetchall()

    return [dict(r) for r in reversed(rows)]
```

```
def get_user_memories(user_id):
    with sqlite3.connect(DB) as con:
        con.row_factory = sqlite3.Row
        rows = con.execute(
            """SELECT memory_key, memory_value, confidence
               FROM user_memories
               WHERE user_id = ?
               ORDER BY updated_at DESC""",
            (user_id,)
        ).fetchall()

    return [dict(r) for r in rows]
```

```
def upsert_memory(user_id, key, value, confidence=1.0):
    with sqlite3.connect(DB) as con:
        con.execute(
            """INSERT INTO user_memories
               (user_id, memory_key, memory_value, confidence)
               VALUES (?, ?, ?, ?)
               ON CONFLICT(user_id, memory_key)
               DO UPDATE SET
                 memory_value=excluded.memory_value,
                 confidence=excluded.confidence,
                 updated_at=CURRENT_TIMESTAMP""",
            (user_id, key, value, confidence)
        )
```

41.1. Tạo memory context cho Agent

```
def build_memory_context(user_id):
    history = load_recent_history(user_id, limit=6)
    memories = get_user_memories(user_id)

    mem_text = "\n".join(
        f"- {m['memory_key']}: {m['memory_value']}"
        for m in memories[:10]
    )

    hist_text = "\n".join(
        f"Q: {h['question']}\nA: {h['answer'][:300]}"
        for h in history
    )

    return f"""THÔNG TIN NGƯỜI DÙNG:
{mem_text}

LỊCH SỬ GẦN ĐÂY:
{hist_text}"""
```

42. Cho Agent tự trích xuất Semantic Memory

Ở phiên bản cơ bản, nhóm có thể chỉ lưu query_logs. Ở phiên bản nâng cao, sau mỗi câu trả lời có thể dùng LLM để trích các thông tin bền vững rồi lưu vào user_memories. Nên dùng allow-list để tránh lưu mọi thứ một cách thiếu kiểm soát.

Prompt trích memory mẫu:

```
MEMORY_EXTRACT_PROMPT = """
Từ hội thoại dưới đây, chỉ trích xuất thông tin BỀN VỮNG
có ích cho các lần tư vấn sau.

Chỉ cho phép các key:
- business_type
- industry
- province
- frequent_topic
- preferred_answer

Không lưu mật khẩu, số định danh, thông tin bí mật hoặc dữ liệu nhạy cảm.
Nếu không có thông tin phù hợp, trả []

Chỉ trả JSON:
[
  {"key": "...", "value": "...", "confidence": 0.0}
]

CÂU HỎI: {question}
TRẢ LỜI: {answer}
"""
```

```
def save_extracted_memories(user_id, items):
    for item in items:
        if item.get("confidence", 0) >= 0.80:
            upsert_memory(
                user_id,
                item["key"],
                item["value"],
                item["confidence"]
            )
```

Khuyến nghị: Để dễ bảo vệ project khi thuyết trình, có thể cho người dùng bật/tắt memory_opt_in. Nếu tắt, hệ thống vẫn đăng nhập và trả lời nhưng không tạo semantic memory mới.

43. Tích hợp Memory vào AgentState và LangGraph

Mở rộng state.py:

```
class AgentState(TypedDict, total=False):
    user_id: int
    session_id: str
    query: str

    memory_context: str
    history: list[dict]

    documents: list[dict]
    db_results: list[dict]
    web_results: list[dict]
    grades: dict
    route: str
    iteration: int
    generation: str
```

Node load_memory:

```
from memory import load_recent_history, build_memory_context

def load_memory(state):
    uid = state["user_id"]
    return {
        "history": load_recent_history(uid, limit=6),
        "memory_context": build_memory_context(uid)
    }
```

Node save_interaction sau khi generate:

```

from memory import log_interaction

def save_interaction(state):
    log_interaction(
        user_id=state["user_id"],
        session_id=state["session_id"],
        question=state["query"],
        answer=state["generation"],
        route=state.get("route", "knowledge"),
        source_type="rag/db/web"
    )
    return state

```

Lưu LangGraph sau khi bổ sung:

```

g.set_entry_point("load_memory")
g.add_edge("load_memory", "router")

# ... RAG / DB / CRAG / web như cũ ...

g.add_edge("generate", "save_interaction")
g.add_edge("save_interaction", "update_memory")
g.add_edge("update_memory", "finalize")

```

44. Prompt trả lời có sử dụng Memory đúng cách

```

ANSWER_WITH_MEMORY = """
Bạn là trợ lý tuân thủ pháp lý doanh nghiệp.

BỐI CẢNH NGƯỜI DÙNG:
{memory_context}

EVIDENCE PHÁP LÝ HIỆN TẠI:
{legal_context}

CÂU HỎI HIỆN TẠI:
{query}

Quy tắc:
1. Memory chỉ dùng để hiểu hoàn cảnh và nhu cầu người dùng.
2. Không coi câu trả lời cũ là căn cứ pháp lý.
3. Mọi kết luận pháp lý phải dựa trên EVIDENCE hiện tại.
4. Ghi số hiệu văn bản, điều khoản và nguồn.
5. Nếu evidence chưa đủ, nói rõ "chưa đủ căn cứ".
"""

```

Cách tách rõ `memory_context` và `legal_context` giúp giảm nguy cơ Agent “nhớ” một câu trả lời cũ đã lỗi thời rồi dùng lại như luật hiện hành.

45. Giao diện Streamlit: Register, Login và lịch sử truy vấn

Giao diện demo chỉ cần đơn giản nhưng nên cho thấy rõ tính multi-user: đăng ký, đăng nhập, lịch sử riêng và memory riêng.

```

import streamlit as st
from auth import create_user, authenticate
from memory import load_recent_history

if "user" not in st.session_state:
    st.session_state.user = None

st.title("Legal CRAG Agent")

if st.session_state.user is None:
    tab1, tab2 = st.tabs(["Đăng nhập", "Đăng ký"])

    with tab1:
        u = st.text_input("Username")
        p = st.text_input("Password", type="password")
        if st.button("Login"):
            user = authenticate(u, p)
            if user:
                st.session_state.user = user
                st.rerun()

```

```

else:
    user = st.session_state.user
    st.success(f"Xin chào {user['username']}")

    question = st.chat_input("Nhập câu hỏi pháp lý...")

    with st.sidebar:
        st.subheader("Lịch sử gần đây")
        for h in load_recent_history(user["id"], 8):
            st.caption(h["question"])

```

Điểm cộng: Có thể thêm nút “Xóa memory của tôi” hoặc “Tắt ghi nhớ” để thể hiện tư duy privacy-by-design trong phần demo.

46. Kịch bản demo người dùng quay lại

Lần dùng	Người dùng hỏi / cung cấp	Agent sử dụng memory
Lần 1	“Tôi đang phụ trách công ty TNHH tại Long An, thường cần tra cứu luật lao động.”	Lưu business_type, province, frequent_topic nếu memory được bật.
Lần 1	“Hợp đồng xác định thời hạn có giới hạn bao lâu?”	RAG/CRAG trả lời từ evidence và log lại câu hỏi.
Lần 2	Đăng nhập lại sau một thời gian.	load_memory nạp profile + một số query gần đây.
Lần 2	“Vậy trường hợp của công ty tôi thì cần lưu ý gì?”	Hiểu “công ty tôi” theo memory nhưng vẫn truy hỏi quy định pháp luật hiện hành trước khi tư vấn.

47. Bảo mật và chống rò rỉ dữ liệu giữa người dùng

- Mọi SELECT lịch sử phải có điều kiện WHERE user_id = ?.
- Không nhận user_id trực tiếp từ ô nhập của người dùng; lấy từ session đăng nhập.
- Password chỉ lưu hash bằng bcrypt.
- Không đưa toàn bộ lịch sử nhiều tháng vào prompt; chỉ lấy Top-N hoặc tóm tắt để giảm token và giảm lộ dữ liệu.
- Không lưu các dữ liệu bí mật/nhạy cảm vào semantic memory nếu không cần thiết.
- Nếu hệ thống có nhiều user, cần test bắt buộc rằng user A không bao giờ đọc được history/memory của user B.
- Nên có memory_opt_in và chức năng xóa memory/log khi phát triển phiên bản thực tế.

Metric an toàn: Tiêu chí “Cross-user leakage = 0%” nên được đưa vào phần thực nghiệm.

48. Đánh giá Memory ở mức Project Thạc sĩ

Ngoài các metric RAGAS, nên bổ sung một nhóm thí nghiệm riêng cho memory:

Metric	Cách đo
History retrieval accuracy	Memory có lấy đúng các lượt trước liên quan tới câu hỏi hiện tại hay không.
Context reuse accuracy	Agent có dùng đúng bối cảnh cũ khi câu hỏi chứa đại từ như “trường hợp của tôi”, “công ty tôi” hay không.
Memory precision	Tỷ lệ semantic memory được lưu thực sự có ích và chính xác.
Cross-user leakage	Số lần dữ liệu user khác xuất hiện / tổng test; mục tiêu = 0%.
Latency overhead	So sánh thời gian trả lời khi bật và tắt memory.
Token/context overhead	Số token tăng thêm do history và semantic memory.

48.1. Thí nghiệm A/B nên làm

Cấu hình	Mô tả
A - No Memory	Agent chỉ nhìn câu hỏi hiện tại.
B - Recent History	Agent nạp 6-8 lượt gần nhất.
C - History + Semantic Memory	Agent nạp lịch sử gần đây + user_memories.

Dùng khoảng 10-15 kịch bản hội thoại nhiều lượt để đo xem cấu hình C có giúp hiểu đúng ngữ cảnh hơn hay không, đồng thời kiểm tra rằng chất lượng grounding pháp lý không giảm.

49. Bổ sung vào báo cáo kỹ thuật

Chương	Nội dung bổ sung
Chương 3 - Thiết kế	3.9 Kiến trúc multi-user; 3.10 Account & Memory database; 3.11 Data flow và isolation theo user_id.
Chương 5 - Xây dựng Agent	5.11 Authentication; 5.12 Episodic Memory; 5.13 Semantic Memory; 5.14 Memory-aware prompt; 5.15 Save/Load memory nodes.
Chương 6 - Thực nghiệm	6.7 No-memory vs history vs semantic-memory; 6.8 Cross-user leakage test; 6.9 Latency/token overhead.
Chương 7 - Thảo luận	7.7 Khi nào memory giúp; 7.8 Failure cases; 7.9 Privacy và dữ liệu lỗi thời.

50. Checklist bổ sung cho Account & Memory

- ☐ Có bảng users và đăng ký/đăng nhập chạy được.
- ☐ Password được hash, không lưu plain text.
- ☐ Có chat_sessions hoặc session_id.
- ☐ Mỗi câu hỏi/câu trả lời được lưu vào query_logs.
- ☐ Có load_recent_history(user_id).
- ☐ Có user_memories để lưu semantic memory.
- ☐ Có memory_context trong AgentState.
- ☐ Có node load_memory trước router.
- ☐ Có node save_interaction sau generate.
- ☐ Có memory_opt_in hoặc cơ chế bật/tắt ghi nhớ.
- ☐ Test user A không đọc được dữ liệu user B.
- ☐ Kết luận pháp lý không lấy query log làm evidence.
- ☐ Có thí nghiệm No Memory vs Memory.

51. Thứ tự triển khai phần Memory

1. Tạo bảng users / query_logs / user_memories
- ↓
2. Viết auth.py và test register/login
- ↓
3. Log mọi query/answer theo user_id
- ↓
4. Nạp recent_history khi user quay lại
- ↓
5. Đưa memory_context vào AgentState
- ↓
6. Test câu hỏi nhiều lượt / đại từ tham chiếu
- ↓
7. Thêm semantic memory có allow-list
- ↓
8. Thêm Streamlit account UI
- ↓
9. Chạy A/B evaluation + privacy test

Trình tự khuyến nghị: Nên hoàn thành query_logs + recent_history trước. Semantic memory tự động là lớp nâng cao, chỉ thêm sau khi multi-user và CRAG cốt lõi đã chạy ổn định.

PHỤ LỤC A - Danh sách file mã nguồn nên có

File	Mục đích
create_db.py	Tạo schema SQLite.
state.py	Định nghĩa AgentState.
llm.py	Cấu hình ChatOllama + embedding.
ingest.py	Chunking và tạo Chroma index.
tools.py	Retriever, database tool, web search.
nodes.py	Các node xử lý của Agent.
graph.py	StateGraph và conditional routing.
main.py	CLI chạy thử.
eval.py	RAGAS và mã đánh giá.
app.py	Giao diện Streamlit - làm sau.
test_set.json	Bộ câu hỏi benchmark.
requirements.txt	Danh sách thư viện.

PHỤ LỤC B - Gợi ý phân chia công việc nhóm

Vai trò	Công việc chính
Thành viên 1 - Data	Thu thập văn bản, làm sạch, metadata, SQLite.
Thành viên 2 - RAG	Chunking, embedding, Chroma, retrieval.
Thành viên 3 - Agent	CRAG grader, LangGraph, web fallback.
Thành viên 4 - Evaluation	Test-set, RAGAS, threshold experiments, LangFuse.
Thành viên 5 - Report/Demo	Streamlit, biểu đồ kết quả, Word/PPT và demo.

PHỤ LỤC C - Tiêu chí tự kiểm tra trước khi nộp

- Có ít nhất một câu in-corpus trả lời đúng và có nguồn.
- Có ít nhất một câu out-of-corpus kích hoạt web fallback.
- Có ít nhất một câu metadata được trả lời từ database.
- Có log/trace thể hiện rõ các node Agent đã đi qua.
- Có bảng so sánh baseline RAG và CRAG.
- Có bộ câu hỏi test và ground-truth.
- Có ít nhất một phân tích failure case.
- Không dùng số liệu đánh giá giả.
- Báo cáo nêu rõ giới hạn và rủi ro của hệ thống pháp lý.

Tài liệu nguồn dùng làm căn cứ

- 1) ĐỀ XUẤT PROJECT MÔN AGENT - 8 đề tài xây dựng AI Agent với LangGraph & Ollama nội bộ.
- 2) HƯỚNG DẪN THỰC HIỆN PROJECT MÔN AGENT - Bản chi tiết: code skeleton, schema DB, test-set và prompt mẫu.